

ASL Letter Classifier

CSE 5819 Term Project

Riley Francis - rif17002



Note: For the most updated version of my code, please see my GitHub repository below. Please see the README in GitHub for instructions on how to run my code.

GitHub repository: <https://github.com/RileyFrancis/ASL-Letter-Classifier>

You can *download my dataset* here if needed:

<https://kaggle.com/datasets/402fce4b3d34f098f00df0805731050351b604bdeb95dd2debc389a4f9bb4ab1>

Translating American Sign Language

Google Translate and other translation tools have existed for about two decades now, but even today there are not really any similar methods of ML-based translation for American Sign Language. The most significant AI ASL translator is SignGemma (developed by Google), which is currently in its testing phase and is scheduled to release by the end of the year. AI ASL translators have a few things that make them more difficult to design. ASL is a visual language with a completely different set of grammar rules than most other languages (the idea that language can exist within 'a space' is difficult to teach an artificial intelligence). Also, because ASL is a visual language, for live translation you would need a model that is capable of processing video input at many frames per second while translating that into the desired language. This is rather computationally expensive, especially for mobile devices where this technology would be used most.

In this project, I have developed a CNN classifier that can identify 24 of the 26 signed letters of the alphabet. It runs fast on the GPU with little to no latency, allowing for live sign classification.

My Methodology and Implementation

To simplify my model and to make data collection vastly easier, I decided to classify only the ASL alphabet instead of other signs. Furthermore, I excluded letters that have any motion, such as the twist of the wrist for J or the zig-zag motion for Z.

The model itself is a convolution neural network (CNN), built using the torch.nn module. The CNN has 9 model layers, and 14 total layers (including Conv, Pool, ReLU, Flatten, Dropout, FC). Here, I use a 3x3 kernel to scan across the image. Below is the Python definition of the CNN that I am using to classify the input images:

```
class CNNClassifier(nn.Module):  
    def __init__(self, num_classes=len(CLASS_NAMES)):  
        super().__init__()
```

PYTHON

```

self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
self.pool1 = nn.MaxPool2d(2, 2)

self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
self.pool2 = nn.MaxPool2d(2, 2)

self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
self.pool3 = nn.MaxPool2d(2, 2)

self.fc1 = nn.Linear(128 * 32 * 32, 256)
self.fc2 = nn.Linear(256, num_classes)

self.dropout = nn.Dropout(0.5)

def forward(self, x):
    x = self.pool1(F.relu(self.conv1(x)))
    x = self.pool2(F.relu(self.conv2(x)))
    x = self.pool3(F.relu(self.conv3(x)))
    x = x.view(x.size(0), -1)
    x = self.dropout(F.relu(self.fc1(x)))
    x = self.fc2(x)

    return x

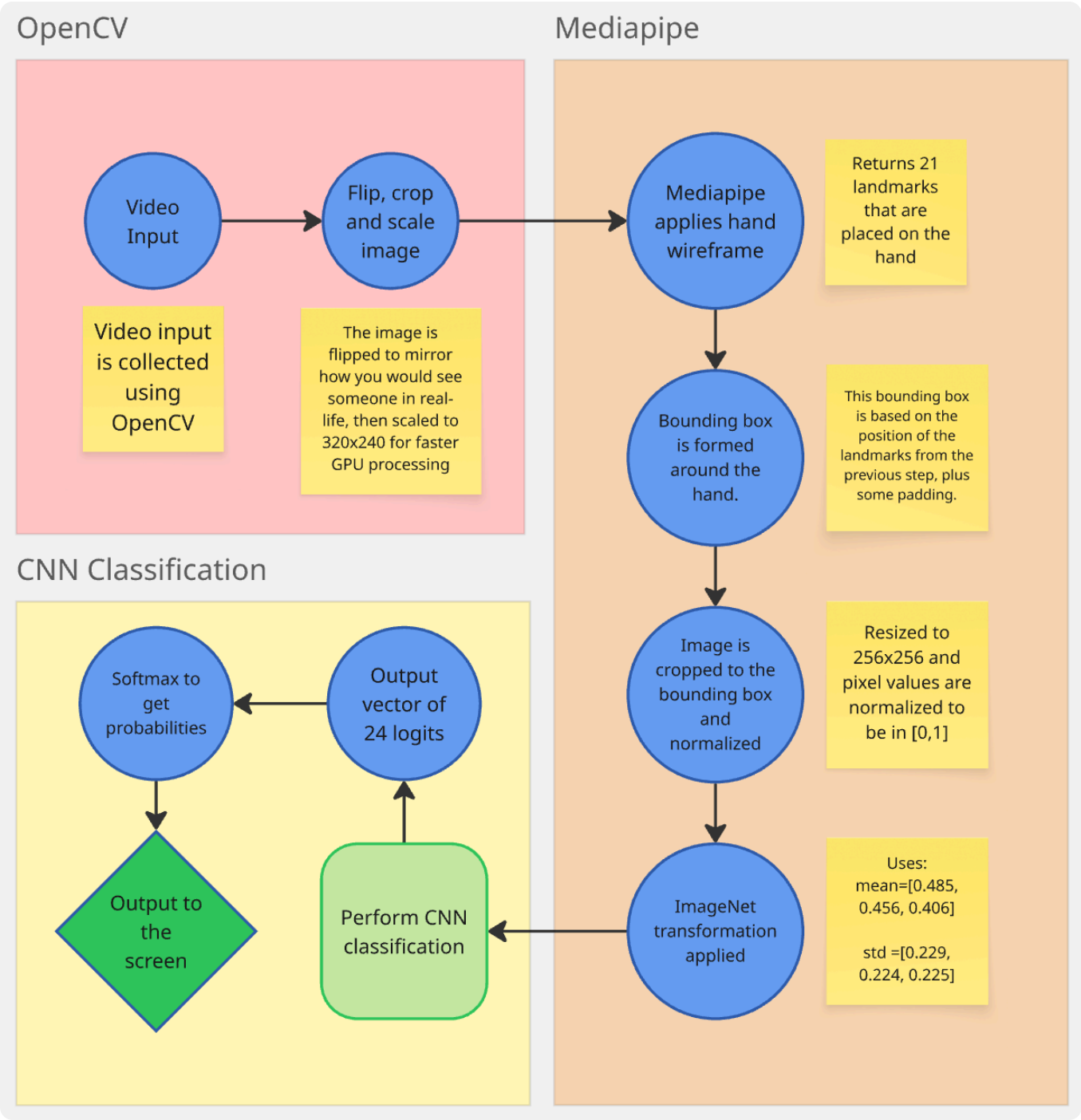
```



I utilize a combination of tools in order to produce a functional image classifier:

- OpenCV's image processing to capture frames from the input video.
- Mediapipe's hands model (developed by Google) to overlay a wireframe model onto the subject's hands for better interpretation by the CNN.
- Torch's Convolution Neural Network module.

Below is a pipeline showing the process of classifying a video frame:



Creating a Dataset

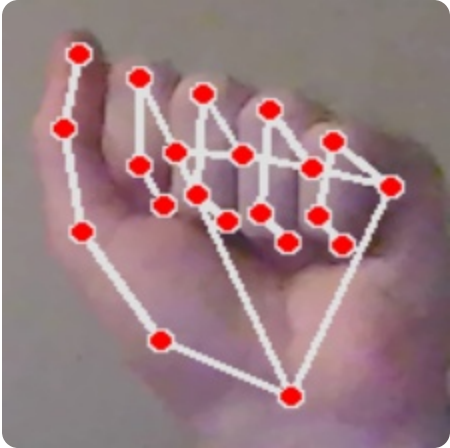
For training my CNN, I decided to create my own dataset because all of the datasets that I found online were not good at all. I recorded myself signing each of the signs that will be classified. In total, there are 39,300 that I recorded, as shown below:

```
Image counts per class:
A      : 1421
B      : 1456
C      : 1302
D      : 1493
E      : 1879
F      : 1222
G      : 2333
H      : 2151
I      : 1931
K      : 2654
L      : 1556
M      : 1282
```

```
N      : 1556
O      : 1411
P      : 1949
Q      : 1562
R      : 1683
S      : 1442
T      : 2136
U      : 1142
V      : 1336
W      : 1313
X      : 1570
Y      : 1520
Total images: 39300
```

You can count the files yourself using `python count_dataset_imgs.py`.

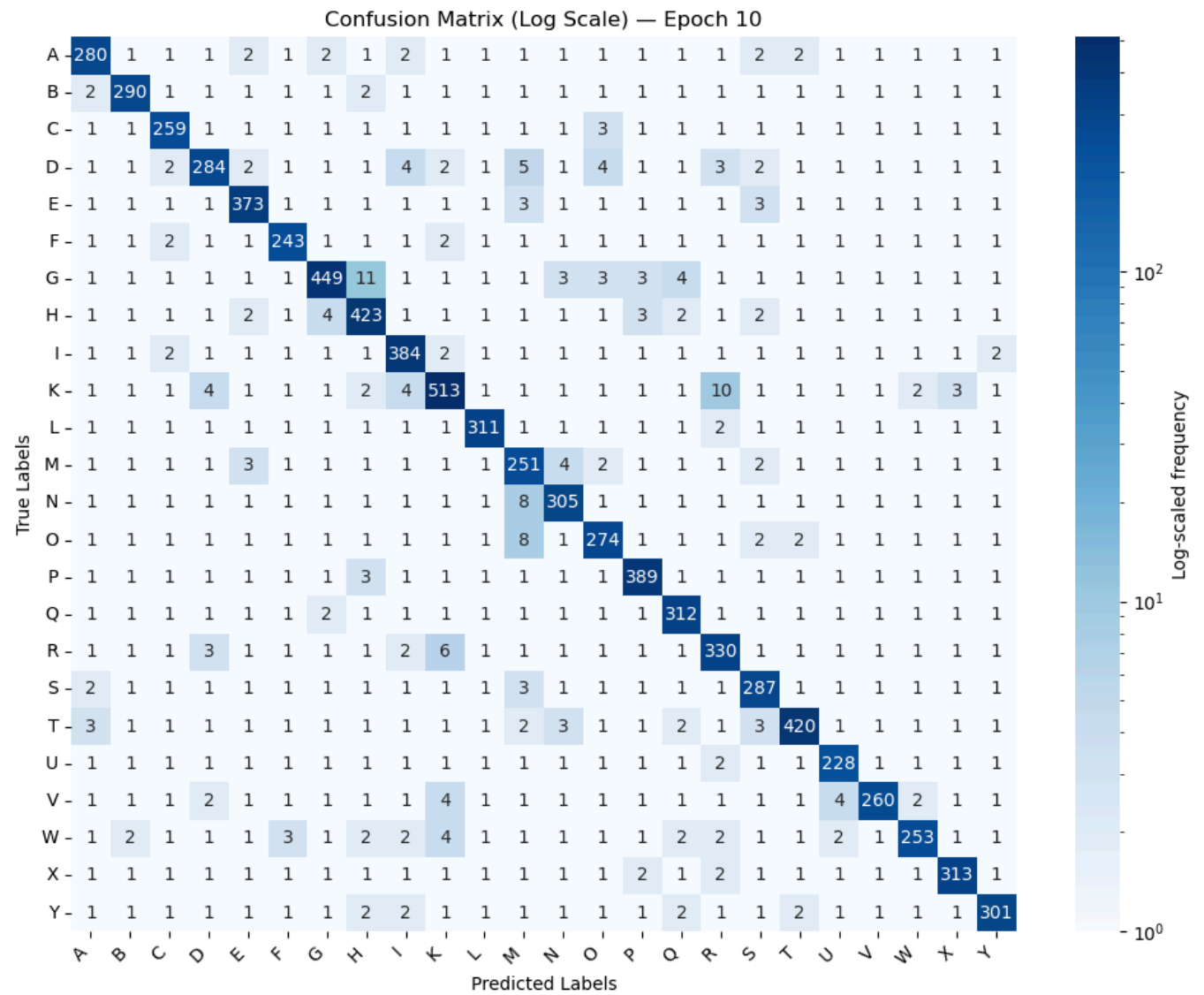
Below you can see an example of a processed image for the 'A' class that's ready for training.
([1762652573956.jpg](#))



Training Results

I used a 20/80 split using `train_test_split` for the test/train data splits.

Below is a confusion matrix after training for 10 epochs. Note that because of the logarithmic scale, to avoid $\log(0)$, I've added 1 to each of the cells.



Below is the training report after 10 epochs:

	precision	recall	f1-score	support
A	0.979	0.996	0.988	284
B	0.997	1.000	0.998	291
C	0.966	0.996	0.981	260
D	0.979	0.926	0.952	299
E	0.989	0.989	0.989	376
F	0.996	0.996	0.996	244
G	0.993	0.955	0.974	467
H	0.979	0.984	0.981	430
I	0.972	0.987	0.979	386
K	0.987	0.966	0.976	531
L	0.997	1.000	0.998	311
M	0.921	0.957	0.939	257
N	0.974	0.971	0.973	311
O	0.961	0.968	0.965	282
P	0.985	0.992	0.989	390
Q	0.978	1.000	0.989	312
R	0.957	0.991	0.974	337
S	0.979	0.986	0.983	289
T	0.998	0.993	0.995	427
U	0.962	0.987	0.974	228
V	1.000	0.978	0.989	267
W	1.000	0.966	0.983	263
X	1.000	0.997	0.998	314
Y	0.997	0.993	0.995	304

accuracy			0.982	7860
macro avg	0.981	0.982	0.982	7860
weighted avg	0.982	0.982	0.982	7860

As you can see, the model is exceeding at classifying the training data. In fact the model is overfitting on the data, meaning that actual live classification accuracy suffers. This is because of the small size and low diversity of the dataset that I've created.

Room for Improvements

Currently my model struggles to correctly classify handshapes with backgrounds or lighting conditions (and probably on people other than me too, though I have not tested that) that it was not trained on. This is largely due to the fact that the dataset that I created is still relatively small, and not nearly diverse enough. This is somewhat fixed by random color and rotational jitters, however to really improve the dataset, it would be better to gather data from far more sources.

Additionally, the data from the location of the mediapipe wireframe is lost when it is flattened onto the image. This data could instead help to train the model.